CS 5470: Compiler Principles and Techniques

Compiler Project Stage 4: IR-TREE GENERATOR
Spring 2010

In this programming assignment, you are to build a IR-tree generator for the MiniJava language. For a given MiniJava input program your IR-tree generator should produce the corresponding list of intermediate representation (IR) trees, representing the input program in an abstract machine language. You are advised to construct the IR-tree generator in three stages.

First, you should produce a list of procedure fragments given the abstract syntax tree (AST) generated by your parser in Stage 2 and type-checked in Stage 3. For each MiniJava method, a procedure fragment contains a representation of the stack frame and an intermediate representation (IR) tree for the method body. Second, you should canonicalize the IR trees produced in Part 1. Third, you should generate a semantically-equivalent C program from the canonical IR trees produced in Part 2. (The translation to C code is simply to make testing of your IR-tree generator feasible.)

This programming assignment is due via electronic submission 11:59pm Friday, March 12.

Part 1: Translate AST to IR trees. In the first part of this programming assignment, you are to produce a list of procedure fragments from an AST. A procedure fragment is represented by the `ProcFrag` class (derived from the `Frag` class) of the `Translate` package. The `ProcFrag` class contains two public field variables: `body` and `frame`. The `body` is a `Tree.Stm` object that represents the method body as an IR tree. The `frame` is a `Frame.Frame` object that represents the method's stack frame. (Note that `MipsFrame` of the `Mips` package is a subclass of `Frame.Frame` specific to the MIPS target.)

The `Translate` class of the `Translate` package implements the `ExpVisitor` of the `visitor` package (i.e., all `visit()` methods return a `Translate.Exp` object). `Translate` visits each node in the AST and constructs an IR tree and stack frame for each method declaration. The IR tree and frame are paired and accumulated in a list of procedure fragments. The provided file `Translate.java` is heavily commented. In this part of the assignment, your job is fill in missing code where indicated.

Part 2: Canonicalize IR trees. In the second part of this programming assignment, you are to simplify the IR trees produced in Part 1 (removing all `Tree.SEQ` and `Tree.ESEQ` nodes, as well as, arranging jumps and labels to take advantage of "fall through"). For each IR tree (rooted at a `Tree.Stm` node `n`),

```
TraceSchedule t = new TraceSchedule(new BasicBlocks(new Canon().linearize(n)))
```

produces a `TraceSchedule` object with a field variable `stms`, which is a list of simplified `Tree.Stm` nodes. In this part of the assignment, your job is to perform this canonicalization for each method body in the accumulated list of procedure fragments. (The `TraceSchedule`, `BasicBlocks`, and `Canon` classes are provided.)

Part 3: Generate C code. In the third part of this programming assignment, you are to generate a C program that is semantically equivalent to the MiniJava input program, using the canonical

IR trees produced in Part 2. For each method and corresponding `ProcFrag p`, `TraceSchedule t` object pair,

```
g.codegen(p, t.stms);
```

produces C code for the method (using the `GenCVisitor g`). To print the entire C program to `System.out`, use the following.

```
g.print();
```

(The `GenCVisitor` class and the associated interface `StringVisitor` are provided in the `IR_visitor` package.)

Output. Many different IR tress can represent the same MiniJava method. Therefore, it is possible (and even expected) that the output you produce for Parts 1-3 will be different from that of the reference compiler and/or your fellow classmates. However, the output of the generated C program may not vary, and must be the same as the output of the input MiniJava program (compiled with `javac`). It is the output of the generated C program that will be used to test your IR-tree generator. Thus, your IR-tree generator must print the generated C program to `System.out` *and nothing else*. The following steps will be used to test the output of the generated C program (which should be the same as the output of the MiniJava input program).

```
java Main < inputMJPrgm.java > equivPrgm.c
gcc -o equivPrgm equivPrgm.c
equivPrgm > equivPrgm.out
diff inputMJPrgm.out equivPrgm.out
```

A subset of the test suite that will be used to grade your IR-tree generator is located on the CADE machines at

`/home/cs5470/project/ir-generator/tests/subset/`

or on the class web pages at

`www.eng.utah.edu/~cs5470/project/generator_tests.zip`.

Notes. The following are a few notes on this programming assignment.

- All of the code mentioned above is in the Compiler Project section of the class web page. See the `README` file for a description of each package, as well as, notes indicating the files you should and should not modify.
- It is a good idea to do some testing after completing each stage of this assignment. Part 1: The `printResults()` method of `Translate.Translate` prints the IR tree representing the body of each procedure fragment accumulated. Part 2: The `prStm(s)` method of `Tree.Print` prints an IR tree rooted at `Tree.Stm s` and can be used to print a canonical IR tree. Part 3: The `print()` method of `IR_visitor.GenCVisitor` prints the semantically-equivalent C program.

Reference Compiler. For your reference, a correct and complete IR-tree generator is provided at

`/home/cs5470/project/ir-generator/ref_compiler/`.

For instructions on how to run the reference IR-tree generator, see the `README` file.

Submission. Turn in a zipped tar file (e.g., `ir-generator.tar.gz`; using `tar -cvfz`) via electronic submission containing

- the parser code for building an AST of the input program (from Stage 2 of the compiler project),
- the semantic analysis code for type-checking the AST (from Stage 3 of the compiler project),
- all files required to implement your IR-tree generator,
- and an Ant file `build.xml` for building and executing the IR-tree generator.

To facilitate testing, the Ant build file's default action should be to build the IR-tree generator (i.e., that is what should happen when you type `ant`). Also, the Ant build file should include a target for executing the IR-tree generator (i.e., that is what should happen when you type `ant run`).

To hand in stage 4 of your compiler project, connect to a CADE machine and issue the command

```
handin cs5470 ir-generator <your gzipfile>
```

Please label all files contained in the zipfile with your name and your partner's name (if you are working in a group).

Do not forget to turn in your project evaluation document in class on March 15.